

UNIVERSITÀ DEGLI STUDI DI VERONA

Elaborato SIS

Architettura degli Elaboratori - Laboratorio

Corso di Laurea in Informatica

Jonathan Fin (VR471297) e Gabriel Adami (VR472400)

24/02/2022

Macchinario chimico per portare il pH ad un valore di neutralità

Indice

Specifiche del progetto	3
Architettura generale del circuito	4
Diagramma degli stati del controllore	5
Architettura del Datapath	7
Statistiche del circuito	8
Mapping tecnologico	10
Scelte progettuali	11

Specifiche del progetto

Il circuito sequenziale che verrà presentato controlla un macchinario chimico il cui scopo è portare una soluzione iniziale a pH noto ad un pH di neutralità. Il valore del pH viene espresso in valori compresi tra 0 e 14.

Il circuito controlla due valvole di erogazione: una di soluzione acida e una di soluzione basica.

Se la soluzione iniziale è acida, il circuito dovrà procedere all'erogazione della soluzione basica fintanto che la soluzione finale non raggiunga la soglia di neutralità (pH compreso tra 7 e 8).

Analogamente, se la soluzione iniziale è basica, il circuito procederà all'erogazione di soluzione acida fino al raggiungimento della soglia di neutralità.

Per pH acido si intende un valore strettamente inferiore a 7, mentre per basico si intende una soluzione con pH strettamente maggiore a 8.

Le due valvole hanno flussi differenti di erogazione.

La valvola relativa alla soluzione basica eroga una quantità di soluzione che permette di alzare il pH della soluzione iniziale di 0.25 ogni ciclo di clock.

La valvola relativa alla soluzione acida eroga una quantità di soluzione che permette di abbassare il pH della soluzione iniziale di 0.5 ogni ciclo di clock.

Il circuito ha 3 ingressi:

- RST [1 bit]
- START [1 bit]
- PH [8 bit, fixed point: 4 parte intera + 4 parte decimale]

Gli output sono i seguenti:

- FINE_OPERAZIONE [1 bit]
- ERRORE_SENSORE [1 bit]
- VALVOLA_ACIDO [1 bit]
- VALVOLA_BASICCO [1 bit]
- PH_FINALE [8 bit]
- NCLK [8 bit]

Input e output sono definiti nell'ordine sopra specificato (da sinistra verso destra).

Il meccanismo è guidato come segue:

- Quando il segnale RST viene alzato, il sistema torna da un qualsiasi stato allo stato di Reset, mettendo tutte le porte in output a zero.
- Per procedere, il sistema riceve in input il segnale di START, con valore 1, e il segnale del pH iniziale per un solo ciclo di clock. Il sistema potrà quindi procedere con la fase di elaborazione.
- Se la soluzione iniziale è acida, viene aperta la valvola della soluzione basica, mettendo a 1 il relativo output. Analogamente, se la soluzione iniziale è basica, viene aperta la valvola della soluzione acida mettendo a 1 la porta VALVOLA_ACIDO.
- Il sistema mantiene aperte le valvole per il tempo necessario al raggiungimento della soglia di neutralità (calcolata dal sistema).
- Una volta terminata l'operazione, il sistema deve chiudere tutte le valvole aperte, riportare il pH finale sulla porta in output PH_FINALE e alzare la porta di FINE_OPERAZIONE.
- La porta NCLK riporta quanti cicli di clock sono stati necessari per portare la soluzione a neutralità.
- Se il valore del pH non è valido (> 14) il sistema deve riportare l'errore alzando l'output ERRORE_SENSORE.

Architettura generale del circuito

Il circuito è composto di una FSM (controllore) e un data-path (elaboratore). I due file (non ottimizzati) che contengono la rappresentazione di queste componenti sono presenti nella cartella allegata con i nomi FSM.blif e datapath.blif. Il file FSMD.blif permette di collegare il controllore e l'elaboratore in modo che le due componenti possano scambiarsi dati e che il sistema complessivo funzioni correttamente.

Di seguito viene riportato lo schema generale del circuito, in cui vengono mostrati inputs, outputs e gli scambi di informazioni tra FSM e data-path

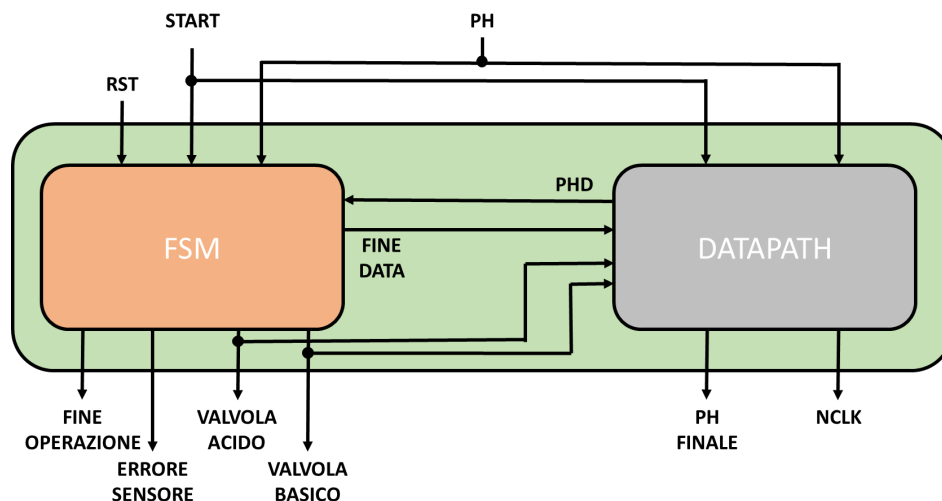


Diagramma degli stati del controllore

Abbiamo realizzato un controllore (FSM) con 5 stati:

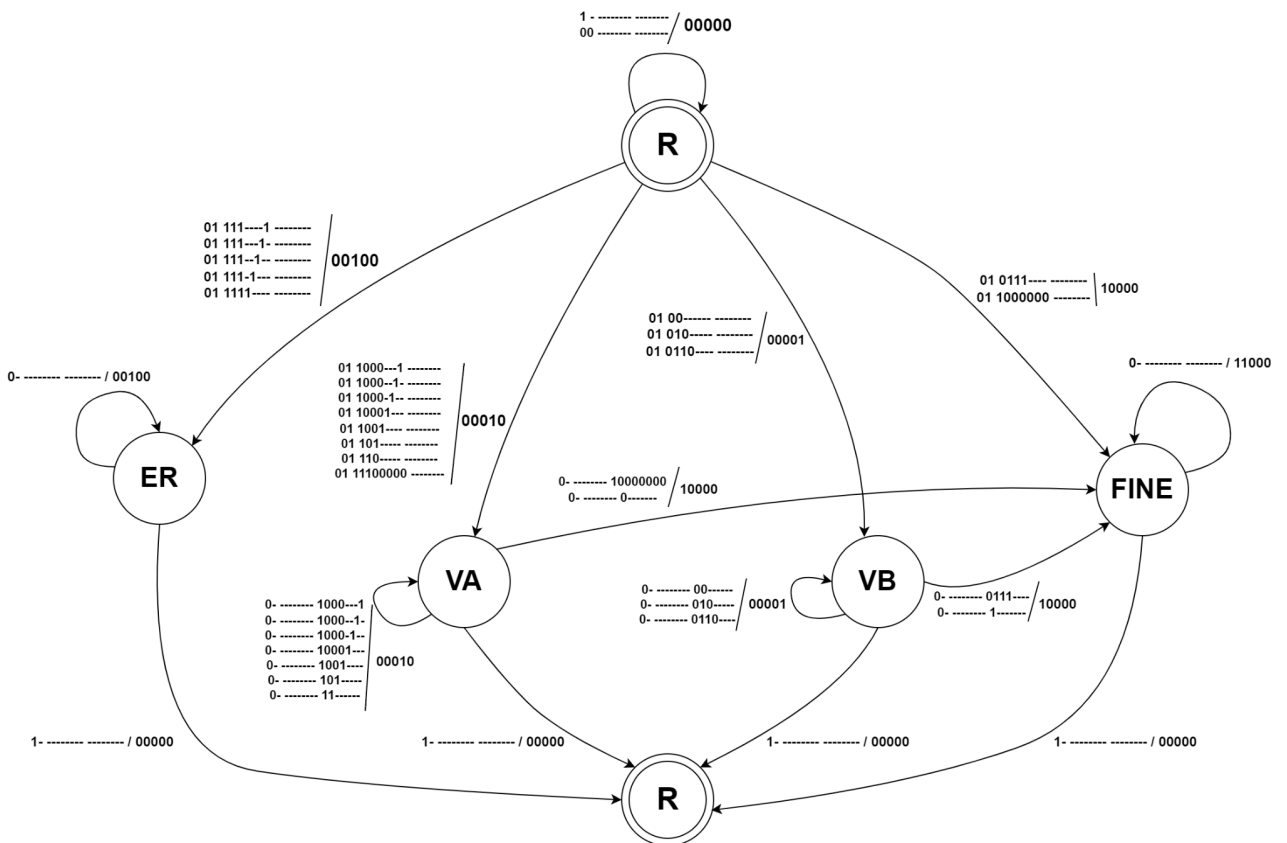
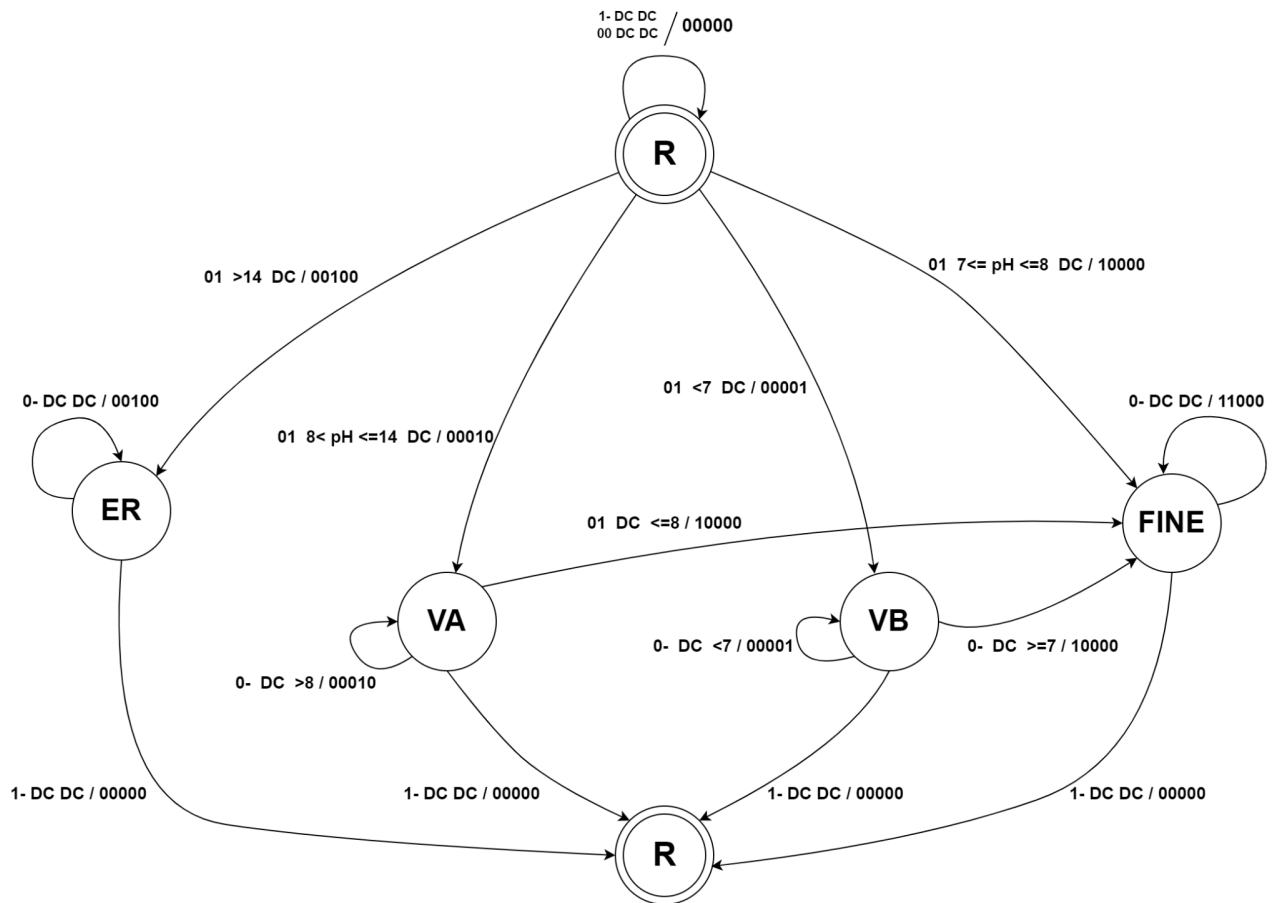
1. R: lo stato iniziale e di reset, che effettua un controllo sul valore del pH iniziale quando START viene posto a 1;
2. ER: lo stato di errore; ci si trova in questo stato quando il valore del pH è strettamente maggiore di 14;
3. VA: ci si trova in questo stato quando il pH è basico e viene quindi aperta la valvola della soluzione acida;
4. VB: ci si trova in questo stato quando il pH è acido e viene quindi aperta la valvola della soluzione basica;
5. FINE: lo stato di fine, nel quale ci si trova quando il pH è neutro e vengono quindi chiuse tutte le valvole e stampati gli output della FSMD.

La nostra FSM prevede 18 inputs e 5 outputs:

- gli inputs sono (in ordine da sinistra verso destra): RST, START, PH [8 bit] (quello iniziale) e PHD [8 bit] (il pH elaborato dal datapath);
- gli outputs sono (tutti a 1 bit): FINE_DATA, FINE_OPERAZIONE, ERRORE_SENSORE, VALVOLA_ACIDO e VALVOLA_BASICCO.

FINE_DATA è un output che, quando il pH diventa neutro, diventa 1 per un solo ciclo di clock. Nel ciclo di clock successivo FINE_DATA è posto a 0 e FINE_OPERAZIONE a 1. Ciò serve per indicare il termine delle operazioni al data-path, il quale al ciclo successivo stamperà i risultati del PH_FINALE e NCLK.

Di seguito riportiamo due FSM (realizzate con diagrams.net): la prima è più facilmente leggibile, in quanto in input vengono riportati i valori del pH in cifre intere, con i simboli di maggiore (>) / minore (<) / uguale (=) (DC = don't care per tutti gli 8 bit); la seconda è quella implementata in SIS, con la codifica binaria.



Architettura del Datapath

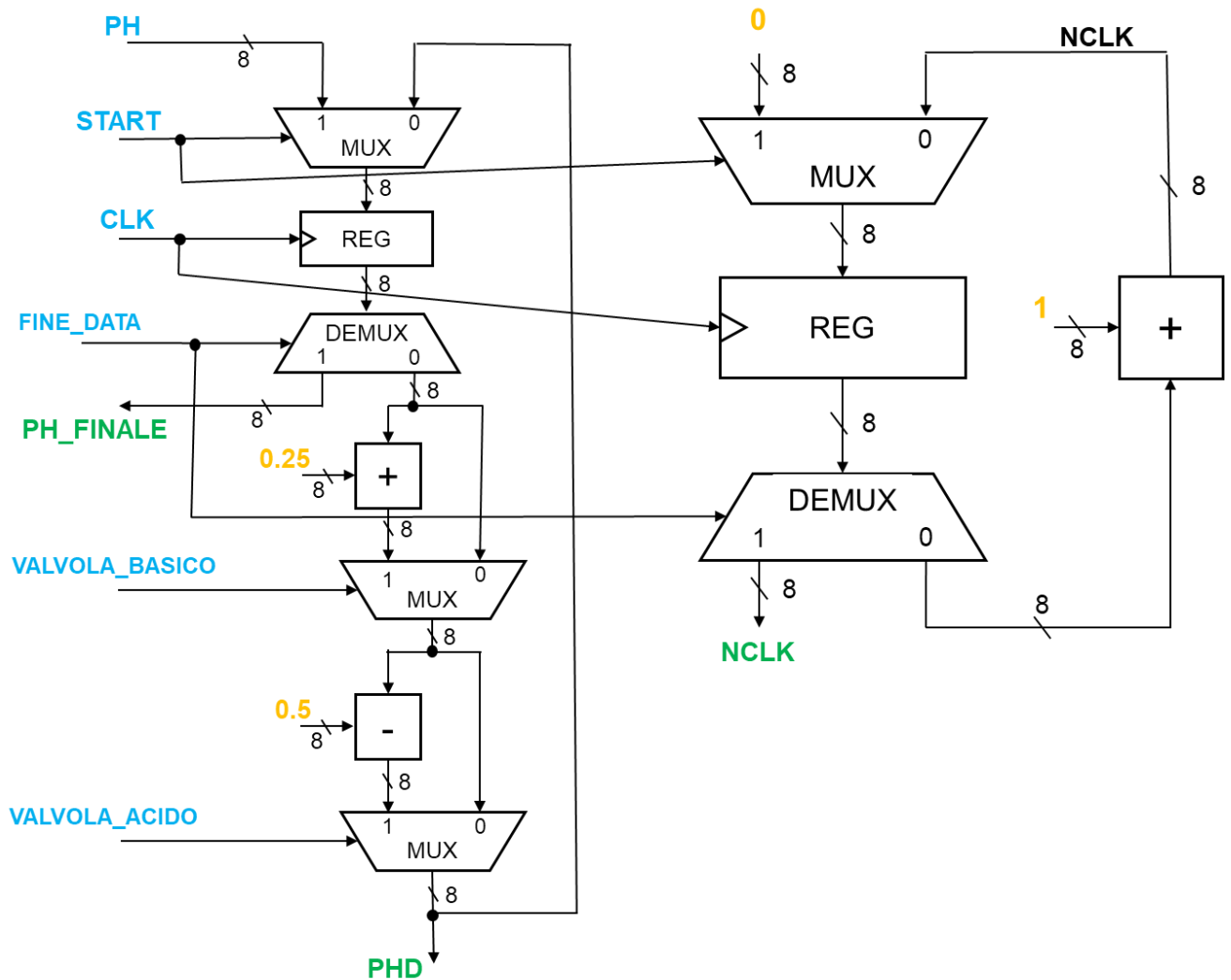
Il nostro data-path (elaboratore del circuito) prevede 12 inputs e 24 outputs:

- gli inputs sono: START, PH (iniziale) [8 bit], FINE_DATA, VALVOLA_ACIDO, VALVOLA_BASIC0;
- gli outputs sono: PHD [8 bit], PH_FINALE [8 bit], NCLK [8 bit]

È stato realizzato utilizzando le seguenti componenti:

- 2 registri a 8 bit (registro_8b.blif): tengono in memoria i valori del PHD e NCLK;
- 2 sommatore a 8 bit (sommatore_8b.blif): utilizzati per incrementare il pH di 0.25 quando viene erogata la soluzione basica e per incrementare di 1 il valore di NCLK ogni ciclo di clock;
- 1 sottrattore (con numeri in modulo) a 8 bit (sottrattore_8b.blif): utilizzato per diminuire il valore del pH di 0.5 quando viene erogata la soluzione acida;
- 2 demultiplexer a 2 uscite a 8 bit (demux_2u_8b.blif): scelgono se stampare gli outputs o continuare le operazioni in base al valore di FINE_DATA (se =1 stampano gli outputs);
- 4 multiplexer a 2 ingressi a 8 bit (mux_2i_8b.blif): 2 mux scelgono gli inputs in base al valore di START, mentre gli altri 2 scelgono quale valore di pH far passare in base ai valori di VALVOLA_BASIC0 e VALVOLA_ACIDO.

In particolare: se START = 1, passano i valori del pH iniziale e viene resettato (posto a 0) il contatore di NCLK, se START = 0, passano i valori del PHD e NCLK; se VALVOLA_BASIC0 = 1 passa il valore del pH aumentato di 0.25, mentre se VALVOLA_ACIDO = 1 passa il valore del pH diminuito di 0.5.



Statistiche del circuito

Statistiche della FSMD prima della minimizzazione:

```
sis> print_stats
FSMD          pi=10   po=20   nodes=169   latches=22
lits(sop)=1210
```

Abbiamo provato a minimizzare il circuito con il comando `source script.rugged`, che in generale è il migliore per ottimizzare un circuito su sis, e in seguito con il comando `fx`, che diminuisce il numero di letterali ma aumenta i nodi.

Abbiamo provato a minimizzare i letterali sia sulla FSM che sul data-path; in seguito riportiamo le statistiche prima e dopo la minimizzazione e i comandi usati.

FSM (con la codifica degli stati):

```
sis> print_stats
FSM          pi=18  po= 5  nodes= 8      latches= 3
lits(sop)= 602  #states(STG)= 5
sis> source script.rugged
sis> source script.rugged
sis> fx
sis> print_stats
FSM          pi=18  po= 5  nodes= 20     latches= 3
lits(sop)= 76  #states(STG)= 5
```

Data-path:

```
sis> print_stats
DATAPATH     pi=12  po=24  nodes=161   latches=16
lits(sop)= 608
sis> source script.rugged
sis> source script.rugged
sis> source script.rugged
sis> source script.rugged
sis> source script.rugged
sis> source script.rugged
sis> fx
sis> print_stats
DATAPATH     pi=12  po=24  nodes= 61   latches=16
lits(sop)= 211
```

Abbiamo utilizzato più volte lo script.rugged in quanto si ottenevano risultati sempre migliori, prima di arrivare ad un punto in cui non si riusciva ad ottimizzare il circuito ulteriormente.

In seguito abbiamo copiato il file non_ottimizzato/FSMD.blif e abbiamo incluso i nuovi file ottimizzati FSM_ott.blif e datapath_ott.blif al posto dei precedenti non ottimizzati; queste sono le statistiche che abbiamo ottenuto:

```
sis> print_stats
FSMD         pi=10  po=20  nodes= 81   latches=22
lits(sop)= 287
```

I nodi e i letterali sono la somma di quelli del datapath e FSM; ai latches se ne aggiungono anche 3 presenti nella FSMD.

L'FSMD può essere ulteriormente ottimizzata:

```
sis> source script.rugged
sis> fx
sis> print_stats
FSMD         pi=10  po=20  nodes= 79   latches=22
lits(sop)= 282
```

Mapping Tecnologico

Dopo l'ottimizzazione del circuito, lo abbiamo mappato per area con la libreria `synch.genlib`. Il nostro circuito presenta le seguenti statistiche:

```
sis> rl FSMD.blif
sis> rlib synch.genlib
sis> map -s 0
warning: unknown latch type at node '{{[5]}}' (RISING_EDGE assumed)
warning: unknown latch type at node '{{[6]}}' (RISING_EDGE assumed)
warning: unknown latch type at node '{{[7]}}' (RISING_EDGE assumed)
WARNING: uses as primary input drive the value (0.20,0.20)
WARNING: uses as primary input arrival the value (0.00,0.00)
WARNING: uses as primary input max load limit the value (999.00)
WARNING: uses as primary output required the value (0.00,0.00)
WARNING: uses as primary output load the value 1.00
>>> before removing serial inverters <<<
# of outputs:      42
total gate area:    6040.00
maximum arrival time: (35.60,35.60)
maximum po slack:    (-3.40,-3.40)
minimum po slack:    (-35.60,-35.60)
total neg slack:     (-589.80,-589.80)
# of failing outputs: 42
>>> before removing parallel inverters <<<
# of outputs:      42
total gate area:    6040.00
maximum arrival time: (35.60,35.60)
maximum po slack:    (-3.40,-3.40)
minimum po slack:    (-35.60,-35.60)
total neg slack:     (-589.80,-589.80)
# of failing outputs: 42
# of outputs:      42
total gate area:    5976.00
maximum arrival time: (35.60,35.60)
maximum po slack:    (-3.40,-3.40)
minimum po slack:    (-35.60,-35.60)
total neg slack:     (-589.20,-589.20)
# of failing outputs: 42
```

L'area totale è quindi 5976.00 e il cammino critico è pari a 35.60.

Scelte progettuali

Nella fsm tra gli stati VA e FINE è stato messo ≤ 8 perché sicuramente non eccederà nel versare la soluzione: da qualsiasi risultato parta, sicuramente con $\text{pH} > 8$, non riuscirà ad avere una soluzione con $\text{pH} < 7$ in un singolo ciclo di clock, in quanto il pH viene abbassato di solo 0.5 per ciclo; accade la stessa cosa, in senso opposto, tra gli stati VB e FINE.

Usiamo FINE_DATA perché il data-path è in ritardo rispetto alla fsm, a causa di alcuni registri posizionati tra FSM e DP. I registri causano il ritardo ma sono indispensabili perché se non ci fossero si genererebbe un *network cycle*, nel quale vengono dati all'infinito valori in input; FINE_DATA serve perché le specifiche del progetto richiedono di avere gli output del DP quando FINE_OPERAZIONE è posto a 1.

Tutti i comandi utilizzati per minimizzare l'FSMD per area sono stati testati più volte e con differenti combinazioni, controllando le statistiche ogni volta (con il comando `print_stats`). Quelli che sono stati riportati nella relazione sono i migliori per ottenere il numero minore di letterali.